검색 품질 개선 (실습)

5-1. 검색이 실패하는 경우들
5-2. 하이브리드 검색 (실습)
5-3. 재랭킹(Re-ranking) (실습)
5-4. 추가로 해볼 만한 것
5-5. 쉬어가기: BM25는 1994년 알고리즘인데 왜 아직 안 밀리나

5-1. 검색이 실패하는 경우들

> 벡터 검색이 실패하는 대표 유형을 직접 재현해봅니다.

RAG 품질 문제의 8할은 생성이 아니라 검색에서 생깁니다. 정답 청크가 검색되지 않으면 LLM이 아무리 좋아도 답할 수 없습니다. 대표적인 실패 유형은 네 가지입니다.

1. 키워드 불일치: 벡터 검색은 의미로 찾지만 반대로 고유명사, 코드번호, 정확한 용어에는 약합니다. "TR-102 양식 어디 있어?" 같은 질문에서 TR-102는 의미가 없는 기호라 벡터가 특별 취급하지 못합니다.
2. 청크 조각화: 정답이 두 청크에 걸쳐 잘린 경우입니다. "3년 이상 근속자는"까지가 청크 1의 끝이라고 해봅니다. 이때 "매 2년마다 1일 가산"이 청크 2의 시작이면 어느 쪽이 검색되어도 반쪽 정보입니다. chunk_overlap이 이를 줄여 주지만 완전히 없애지는 못합니다.
3. 넓은 질문: "회사 규정 요약해줘" 같은 질문은 특정 청크와 비슷하지 않아 k개 안에 핵심이 고루 들어가지 못합니다. MMR이 부분 해결책입니다.
4. 모호한 질문: "그거 신청 어떻게 해?"는 그거가 무엇인지 검색기가 알 수 없습니다. 질문 재작성이 해결책이며 오늘 5-4의 멀티 쿼리 Retriever에서 다룹니다.

따라하기

1. 아래 코드를 `failures.py`로 저장하고 실행해 실패 유형을 재현합니다.

```
# failures.py — 검색 실패 재현
from rag_chain import db

fail_queries = [
    "출장 신청서 TR-102 어디서 내?",   # 유형 1: 고유 기호
    "3년차 연차 가산 규정",             # 유형 2: 청크 경계에 걸릴 수 있음
    "그거 신청 어떻게 해?",             # 유형 4: 모호한 질문
]
for q in fail_queries:
    hits = db.similarity_search_with_score(q, k=2)
    print(f"\n질문: {q}")
    for doc, score in hits:
        print(f"  (거리 {score:.3f}) [{doc.metadata['source']}] {doc.page_con
tent[:40]}")
```

2. 거리 값을 관찰합니다. 거리 값이 평소보다 훨씬 큰 청크만 나온다면 가까운 것이 없어서 그나마 덜 먼 것을 가져온 상태입니다.

벡터 검색은 가까운 것이 없어도 항상 k개를 반환합니다. 결과가 나왔다고 검색이 성공한 것은 아닙니다.

확인: 실패 유형 질문에서 거리 값이 정상 질문보다 커지거나 엉뚱한 파일이 검색되는 것을 보면 성공입니다.

> 벡터 검색은 항상 k개를 반환하므로 결과가 나왔다는 것과 검색이 성공했다는 것은 다르다는 걸 확인했습니다.

5-2. 하이브리드 검색 (실습)

> 미리 구성된 BM25와 벡터 결합 검색의 가중치를 조정하며 효과를 비교해봅니다.

키워드 불일치 유형의 해결책이 하이브리드 검색입니다. 두 검색기의 장단점이 정확히 반대이기 때문에 합치면 서로를 보완합니다.

비교	BM25 (키워드 검색)	벡터 검색
원리	단어 일치 빈도 기반 점수 (검색엔진의 고전 알고리즘)	임베딩 벡터 거리
강점	고유명사, 코드, 정확한 용어	동의어, 의역, 의미 유사
약점	단어가 다르면 실패 (월차와 연차를 다른 단어로 취급)	TR-102 같은 기호에 둔감
비용	무료 (로컬 계산)	임베딩 API 비용

LangChain의 `EnsembleRetriever`가 두 검색기의 결과를 RRF(Reciprocal Rank Fusion, 순위 기반 점수 합산) 방식으로 합칩니다.
각 검색기가 매긴 순위를 점수로 바꿔 합산하므로 두 검색기의 점수 단위가 달라도 공정하게 섞입니다.
`weights`로 어느 쪽을 더 믿을지 정합니다.

BM25는 문장을 단어로 쪼개는 토크나이저가 필요한데 기본값이 공백 분리입니다.

한국어에는 이 기본값이 맞지 않습니다. 조사가 붙어 "연차는", "연차를", "연차가"가 전부 다른 단어로 취급되기 때문입니다. 그래서 형태소 분석기(kiwipiepy)로 명사와 어간 단위로 쪼갭니다.

아래 토크나이저 함수는 사전 검증본으로 제공되므로 그대로 씁니다. 이번 실습에서 직접 만드는 부분은 가중치 조정입니다.

따라하기

1. 미리 구성된 하이브리드 검색 코드를 `hybrid.py`로 저장하고 실행해 세 검색기를 비교합니다.

```
# hybrid.py – 하이브리드 검색 (BM25 구성은 사전 검증된 완성 코드)
from dotenv import load_dotenv
from langchain_community.retrievers import BM25Retriever
from langchain_classic.retrievers import EnsembleRetriever
from kiwipiepy import Kiwi
from metadata import load_chunks
from rag_chain import db

load_dotenv()
kiwi = Kiwi()


def korean_tokenizer(text: str) -> list[str]:
    """한국어 형태소 단위 토크나이저 (사전 검증본). 명사, 어간, 숫자, 외래어만 남깁니다."""
    keep = {"NNG", "NNP", "NNB", "NR", "NP", "VV", "VA", "SL", "SN"}
    return [t.form for t in kiwi.tokenize(text) if t.tag in keep]


# 1) BM25 검색기 – 벡터 DB가 아니라 청크 리스트에서 직접 만듭니다
bm25 = BM25Retriever.from_documents(load_chunks(), preprocess_func=korean_to
kenizer)
bm25.k = 3

# 2) 벡터 검색기
vector = db.as_retriever(search_kwargs={"k": 3})

# 3) 하이브리드 – 가중치 조합 [BM25 비중, 벡터 비중]
hybrid = EnsembleRetriever(retrievers=[bm25, vector], weights=[0.5, 0.5])

queries = [
    "월차 며칠 받아?",         # 의미형 질문 -> 벡터가 유리
    "TR-102 양식 규정",        # 키워드형 질문 -> BM25가 유리
]
for q in queries:
    print("=" * 55)
    print(f"질문: {q}")
    for name, r in [("BM25만", bm25), ("벡터만", vector), ("하이브리드", hybri
d)]:
        hits = r.invoke(q)
        srcs = [h.metadata["source"].split("/")[-1] for h in hits[:3]]
        print(f"  {name}: {srcs}")
```

2. 가중치를 바꿔가며 어느 조합이 이 문서에 맞는지 실험합니다. 아래 코드는 `hybrid.py` 끝에 이어 붙여 실행합니다.
   별도 파일 `hybrid_weights.py` 로 저장한다면 맨 위에 `from langchain_classic.retrievers import EnsembleRetriever` 와 `from hybrid import bm25, vector` 두 줄을 추가합니다.

   ```
   # hybrid_weights.py — 가중치 조정 실험 (hybrid.py에 이어서)
   for w in [0.2, 0.5, 0.8]:
       h = EnsembleRetriever(retrievers=[bm25, vector], weights=[w, 1 - w])
       top = h.invoke("월차 며칠 받아?")[0].metadata["source"].split("/")[-1]
       print(f"BM25 {w:.1f} / 벡터 {1 - w:.1f} -> 1위: {top}")
   ```

3. 가중치 조정 기준입니다.
   - 출발점은 0.5 대 0.5입니다.
   - 양식번호, 제품코드, 고유명사가 많은 질문이면 BM25 비중을 올립니다.
   - 구어체 의역 질문이 많으면 벡터 비중을 올립니다.
   - 최종 판단은 6번째 세션에서 만들 평가 세트의 통과율로 합니다.

막힐 때

- BM25가 디스크의 벡터 DB를 못 읽습니다.

  BM25Retriever는 벡터 DB를 쓰지 않고 메모리에 든 청크 리스트로 만듭니다. 그래서 2번째 세션 2-3에서 만든 `load_chunks()` 함수가 필요합니다.

- `preprocess_func` 를 빼먹었는데 오류가 안 납니다.

  오류 없이 돌아가지만 한국어 성능이 조용히 나빠집니다. 돌아간다와 잘 된다는 다릅니다.

- 첫 실행이 몇 초 멈춰 있습니다: Kiwi()가 사전을 로드하는 시간입니다. 전역에서 한 번만 생성합니다.

- 하이브리드 결과가 3개보다 많이 나옵니다.

  두 검색기 결과의 합집합이라 3-6개가 나올 수 있습니다. LLM에 넣기 전에 상위 n개로 자르는 처리를 넣으면 좋습니다.

확인: 의미형 질문에서는 벡터가 정답 파일을 1위로 올리고 양식번호 질문에서는 BM25가 1위로 올리며 하이브리드가 양쪽 모두에서 정답을 상위에 유지하면 성공입니다.

> BM25와 벡터 검색을 가중치로 결합하면 서로의 약점을 보완한다는 걸 확인했습니다.

5-3. 재랭킹(Re-ranking) (실습)

> 검색 결과를 정밀 모델로 재정렬하면 정확도가 오르는 것을 직접 확인해봅니다.

재랭킹은 검색을 두 단계로 나누는 전략입니다.

1. 1차 검색(빠르고 넓게): 하이브리드 검색으로 후보를 넉넉히 가져옵니다.
2. 재랭킹(느리지만 정확하게): 더 정밀한 모델이 질문과 청크 쌍을 하나씩 직접 비교해 다시 정렬하고 상위만 남깁니다.

두 단계는 관련도를 재는 방식이 다릅니다. 1차 검색의 임베딩은 질문과 문서를 따로 벡터로 만들어 거리만 잽니다. 반면 재랭커(Cross-Encoder)는 질문과 문서를 한 입력으로 같이 읽고 관련도 점수를 냅니다.

같이 읽는 쪽이 훨씬 정확합니다. 다만 후보마다 모델을 한 번씩 돌려야 해서 느립니다. 그래서 넓게 긁고 정밀하게 거른다는 2단 구조를 씁니다.

재랭킹 모델은 패키지를 하나 더 설치해야 씁니다. 모델 파일은 처음 실행할 때 내려받으므로 몇 분 걸릴 수 있습니다.

```
python -m pip install sentence-transformers
```

설치가 끝나면 아래 코드를 `rerank_demo.py`로 저장하고 실행해 재랭킹 전후로 순위가 어떻게 바뀌는지 봅니다.

```
# rerank_demo.py - 재랭킹 전후 순위 비교
from sentence_transformers import CrossEncoder
from hybrid import hybrid

# 다국어 재랭킹 모델 (한국어 지원, 최초 실행 시 다운로드)
reranker = CrossEncoder("BAAI/bge-reranker-v2-m3")

query = "연차 이월 규정"
candidates = [d.page_content for d in hybrid.invoke(query)]  # 1차 검색 후보

scores = reranker.predict([(query, c) for c in candidates])
ranked = sorted(zip(scores, candidates), reverse=True)
for s, c in ranked[:3]:
    print(f"{s:.3f}  {c[:50]}...")
```

1차 검색에서 2-3위였던 청크가 재랭킹 후 1위로 올라오는지 봅니다. 실제 사례에서는 Cohere Rerank API나 Bedrock의 Rerank 모델처럼 API형 재랭커를 쓰는 경우가 많습니다.

오늘은 이런 단계가 존재하고 순위가 실제로 바뀐다는 것까지 알면 충분합니다.

> 넓게 검색한 후보를 정밀 모델로 다시 정렬하는 2단 구조를 확인했습니다.

5-4. 추가로 해볼 만한 것

▼ 멀티 쿼리 Retriever 직접 적용

- 멀티 쿼리 Retriever를 적용해 봅니다. 질문 하나를 LLM이 여러 각도의 질문으로 바꿔 각각 검색한 뒤 결과를 합치는 기법입니다. 넓거나 모호한 질문에 효과가 있습니다. 검색마다 LLM 호출이 추가되므로 비용과 지연이 늘어나는 트레이드오프가 있습니다.

```
# extra_multiquery.py — 멀티 쿼리 Retriever
import logging
from langchain_classic.retrievers.multi_query import MultiQueryRetriever
from rag_chain import db, llm

logging.basicConfig()
logging.getLogger("langchain_classic.retrievers.multi_query").setLevel(logging.
INFO)

mq = MultiQueryRetriever.from_llm(
    retriever=db.as_retriever(search_kwargs={"k": 3}), llm=llm
)
docs = mq.invoke("쉬는 것 관련 규정 알려줘")
print([d.metadata["source"] for d in docs])
```

- 실행하면 로그에 변형 질문이 생성되는 것이 보입니다. 어떤 질문들이 만들어졌는지 관찰해 봅니다.
- 5-2의 하이브리드 검색기를 `rag_chain.py`의 retriever 자리에 끼워 넣고 `eval.py` 통과율이 어떻게 달라지는지 비교해 봅니다.
- 평가 질문을 10개로 늘려 봅니다. fact, paraphrase, no_answer 세 유형의 비율을 기록해 두면 7일차 평가 실습에서 유형별 약점을 분석할 수 있습니다.

5-5. 쉬어가기: BM25는 1994년 알고리즘인데 왜 아직 안 밀리나

> 서른 해 된 키워드 검색이 벡터 시대에도 기본값으로 남은 이유를 봅니다.

BM25는 1994년 TREC 실험에서 정리된 랭킹 함수입니다. 서른 해가 지났는데 아직도 검색엔진 기본값 자리를 지키고 있습니다.

아이디어는 의외로 단순합니다. 희귀한 단어에 가중치를 더 주고, 같은 단어가 열 번 나와도 두 번 나온 문서의 다섯 배로 쳐 주지 않습니다.

살아남은 진짜 이유는 벡터가 못 하는 일을 대신하기 때문입니다. TR-102나 A-1001 같은 식별자는 의미가 없는 기호라 임베딩이 특별 취급하지 못합니다. BM25는 글자가 맞으면 그냥 올려 줍니다.

그래서 상용 RAG는 대부분 하이브리드로 갑니다. Elasticsearch, OpenSearch, Azure AI Search 모두 BM25와 벡터를 함께 물려놨습니다.

5-2에서 쓴 RRF가 합리적인 이유도 같은 맥락입니다. 두 검색기의 점수는 단위가 달라 그냥 더할 수 없습니다. 순위만 쓰면 단위 문제가 사라지니 성격이 다른 검색기도 공정하게 섞입니다.

> BM25가 기호와 고유명사를 붙잡아 주기 때문에 하이브리드가 기본값이 됐다는 걸 확인했습니다.
